ChatGPT

ICNN Stable Dynamics Implementation Report

Executive summary

The stable-dynamics method is best understood as a continuous-time vector-field learner with a built-in Lyapunov projection. The paper defines a nominal dynamics model $\hat{f}(x)$, a scalar Lyapunov function $V(x)$, and then replaces $\hat{f}$ by a closed-form projection

$$f(x) = \hat{f}(x) - \nabla V(x) \frac{\mathrm{ReLU}\Big(\nabla V(x)^\top \hat{f}(x) + \alpha V(x)\Big)}{\|\nabla V(x)\|_2^2},$$

which guarantees $\dot{V}(x) \le -\alpha V(x)$. With a positive-definite, continuously differentiable V that has no nonzero local optima, the paper proves global exponential stability; the companion code implements this projection directly in `Dynamics.forward`. [1]

For a modular Python 3.10+ implementation, the cleanest design is to separate five concerns: physical-system simulation, dataset generation, Lyapunov/ICNN modeling, training, and evaluation. That mirrors the source pipeline—state-space derivative learning with external numerical integration—while making it easy to swap systems, parameters, model families, and losses. The original repository already separates `datasets`, `models`, `train.py`, and `pendulum_error.py`, but it also mixes in notebooks, shell wrappers, and a video-texture/VAE branch that is irrelevant to a physical-systems-only redesign. [2]

The main implementation caveat is that the public code is broader than the paper and not always paper-faithful. The manuscript describes an ICNN-based $V(x) = \sigma(g(x) - g(0)) + \varepsilon\|x\|_2^2$ with smoothed ReLU; the repository exposes several variants, and its public sweep script defaults to `NN-REHU`, which is not a strict ICNN. Public issues also note that the exact pendulum reproduction hyperparameters were never fully documented, and the question of which Lyapunov parameterization produced the paper's results remains unresolved in the public thread. For that reason, I recommend shipping two modes: a **paper-faithful stable ICNN** as the default, and **repo-reproduction ablations** as optional baselines. [3]

Source-grounded reconstruction

The mathematical core

The paper studies autonomous continuous-time systems $\dot{x}(t) = f(x(t))$ and constructs stability by jointly learning a nominal vector field $\hat{f}$ and a Lyapunov function V. The Lyapunov branch is built from an input-convex neural network, using positive hidden-to-hidden weights and convex nondecreasing activations so that the scalar function $g(x)$ is convex. Positive definiteness is then enforced by shifting to the origin and adding a small quadratic term, while continuous differentiability is enforced with a smoothed ReLU that is quadratic on $(0, d)$ and satisfies $\sigma(0) = 0$. The paper also allows an optional differentiable invertible warp F before the ICNN, but the invertibility requirement matters: arbitrary feature engineering is not theorem-preserving. [4]

The repository maps those ideas into a compact PyTorch implementation. `stabledynamics.py` defines the nominal MLP `fhat`, the projection wrapper `Dynamics`, several Lyapunov parameterizations (`ICNN`, `MakePSD`, `PosDefICNN`, `ActualPendulumEnergy`), and an optional auxiliary loss term `smooth_v`; `train.py` then optimizes the model with Adam on batches of state/derivative pairs. For pendulum experiments, the dataset generator builds symbolic equations with Kane's Method in SymPy and caches derivative pairs in `pendulum-cache`, while evaluation is done by numerically rolling out both the true simulator and the learned model with RK4. [5]

Model-family choices

Choice	What it is	Stability status	When to use
Baseline MLP	Plain derivative model for $\dot{x}$ with no Lyapunov branch	No guarantee	Sanity baseline
Paper-faithful stable ICNN	MLP $\hat{f}$ + true ICNN g + live $g(0)$ subtraction + smoothed outer activation + quadratic term + projection	Strongest match to theorem	Default
Repo `PSICNN`	`PosDefICNN` plus projection	Practical variant	Useful ablation
Repo `PSD-REHU`	`MakePSD(ICNN(..., activation=ReHU))` plus projection	Close to manuscript spirit, but source code still needs a fix for live $g(0)$	Reproduction ablation
Repo `NN-REHU`	`MakePSD(MLP(...))` plus projection	Positive-definite wrapper, not ICNN-based	Reproduction-only
Known-energy oracle	Analytic energy as V	Strong prior	Debugging and upper-bound baseline

The repository exposes the variant family directly in `configure(props)`, and the shell sweep chooses `NN-REHU`; the paper, by contrast, describes an ICNN-based construction. That mismatch is exactly the subject of a public issue. [6]

A second source-level caveat matters enough to change the implementation: `MakePSD` stores `self.zero = f(torch.zeros(1,n))` once during initialization, while the paper's formula uses the **current** $g(0)$ during training. In a modern reimplementation, `g(0)` should be recomputed on every forward pass or enforced structurally through a bias-free design; otherwise, you are no longer actually implementing $g(x) - g(0)$ from the manuscript. I therefore treat **live** $g(0)$ as mandatory in the default implementation. The same caution applies to `EndPSICNN`: the paper's optional warp requires an invertible F, while the repository's convenience stack does not visibly enforce invertibility. [7]

Recommended project structure

The repository shows the right conceptual split—datasets, models, training, and evaluation—but the public tree is still optimized for quick experimentation rather than maintainable research software. A Python 3.10+ redesign should keep the same conceptual boundaries, remove the video-texture branch, and replace shell-string module loading with typed configs and package imports. The physical-systems-only path can also drop legacy image/video dependencies from the original requirements, because the pendulum/stable-dynamics core uses torch, numpy, scipy, sympy, and matplotlib rather than the VAE/image stack. [8]

```
stable_icnn_dynamics/
├── pyproject.toml
├── README.md
├── configs/
│   ├── systems/
│   │   ├── pendulum.yaml
│   │   ├── n_link_pendulum.yaml
│   │   └── mass_spring_damper.yaml
│   ├── models/
│   │   ├── stable_icnn.yaml
│   │   ├── baseline_mlp.yaml
│   │   └── known_energy.yaml
│   ├── train/
│   │   ├── paper_faithful.yaml
│   │   ├── repo_reproduction.yaml
│   │   └── modern_default.yaml
│   └── eval/
│       └── default.yaml
├── src/stable_dyn/
│   ├── config.py
│   ├── registry.py
│   ├── systems/
│   │   ├── base.py
│   │   ├── pendulum.py
│   │   ├── n_link_pendulum_sympy.py
│   │   └── mass_spring_damper.py
│   ├── sim/
│   │   ├── integrators.py
│   │   └── simulate.py
│   ├── data/
│   │   ├── schema.py
│   │   ├── build_dataset.py
│   │   └── io.py
│   ├── models/
│   │   ├── activations.py
│   │   ├── icnn.py
```

```
│   │   ├── lyapunov.py
│   │   ├── stable_dynamics.py
│   │   ├── baselines.py
│   │   └── known_energy.py
│   ├── training/
│   │   ├── losses.py
│   │   ├── engine.py
│   │   └── callbacks.py
│   ├── eval/
│   │   ├── metrics.py
│   │   └── plots.py
│   ├── scripts/
│   │   ├── generate_dataset.py
│   │   ├── train.py
│   │   └── evaluate.py
│   └── utils/
│       ├── seeding.py
│       ├── checkpointing.py
│       └── logging.py
├── notebooks/
│   ├── simulate_systems.ipynb
│   ├── build_datasets.ipynb
│   ├── train_models.ipynb
│   └── evaluate_models.ipynb
└── tests/
    ├── test_systems.py
    ├── test_integrators.py
    ├── test_icnn.py
    ├── test_lyapunov.py
    ├── test_stable_projection.py
    └── test_training_step.py
```

Below is the requested mermaid diagram for module interactions.

```
flowchart LR
    A[configs] --> B[systems]
    B --> C[sim/simulate.py]
    C --> D[data/build_dataset.py]
    D --> E[data/*.npz or *.json]
    E --> F[training/engine.py]
    F --> G[models/stable_dynamics.py]
    G --> H[eval/metrics.py]
    G --> I[eval/plots.py]
    H --> J[reports/metrics.json]
    I --> K[reports/plots/*.png]
```

```
    L[notebooks] --> B
    L --> D
    L --> F
    L --> H
    L --> I
```

Module blueprints

The code below implements the paper's projection formula, preserves the repository's continuous-time/RK4 workflow, and makes system swapping explicit. The most important design choice is that notebooks should call **package code**, not duplicate logic. [9]

Systems and simulation

The source benchmark is an n-link damped pendulum generated symbolically; I keep that option, but for a clean starter package I also recommend two analytic systems: a single-link damped pendulum and a mass-spring-damper system. The mass-spring-damper example is an added assumption because you asked for at least two physical systems; the source benchmark itself is pendulum-only. [10]

```python
# src/stable_dyn/systems/base.py
from __future__ import annotations

from dataclasses import dataclass
from typing import Protocol
import numpy as np


class ContinuousSystem(Protocol):
    state_dim: int
    def rhs(self, t: float, x: np.ndarray) -> np.ndarray: ...


@dataclass
class PendulumConfig:
    g: float = 9.81
    length: float = 1.0
    damping: float = 0.3

@dataclass
class MassSpringDamperConfig:
    mass: float = 1.0
    spring_k: float = 2.0
    damping_c: float = 0.4


# src/stable_dyn/systems/pendulum.py
```

```python
class DampedPendulum:
    """State x = [theta, omega]. Stable equilibrium at theta = 0."""
    state_dim = 2

    def __init__(self, cfg: PendulumConfig):
        self.cfg = cfg

    def rhs(self, t: float, x: np.ndarray) -> np.ndarray:
        theta, omega = x[..., 0], x[..., 1]
        dtheta = omega
        domega = -(self.cfg.damping * omega) - (self.cfg.g / self.cfg.length) *
np.sin(theta)
        return np.stack([dtheta, domega], axis=-1)


# src/stable_dyn/systems/mass_spring_damper.py
class MassSpringDamper:
    """State x = [position, velocity]."""
    state_dim = 2

    def __init__(self, cfg: MassSpringDamperConfig):
        self.cfg = cfg

    def rhs(self, t: float, x: np.ndarray) -> np.ndarray:
        q, v = x[..., 0], x[..., 1]
        dq = v
        dv = -(self.cfg.spring_k / self.cfg.mass) * q - (self.cfg.damping_c /
self.cfg.mass) * v
        return np.stack([dq, dv], axis=-1)
```

For the paper-faithful pendulum benchmark, the symbolic builder should mirror the repository's Kane's Method flow rather than attempting to hand-derive the n-link equations:

```python
# src/stable_dyn/systems/n_link_pendulum_sympy.py
def build_n_link_pendulum_rhs(n: int, lengths=None, masses=1.0, friction=0.3):
    """
    Pseudocode matching the repository:
      1. Create q_i, u_i, m_i, l_i symbols
      2. Build frames and particles iteratively
      3. Use Kane's Method to obtain mass_matrix_full and forcing_full
      4. lambdify both expressions
      5. At runtime, solve M(q, u) * xdot = f(q, u)
    """
    ...
```

Use RK4 as the default integrator, because the repository evaluates both true and learned pendulum rollouts with RK4 at fixed step size 0.01. [11]

```
# src/stable_dyn/sim/integrators.py
from __future__ import annotations
import numpy as np

def rk4_step(rhs, t: float, x: np.ndarray, dt: float) -> np.ndarray:
    k1 = rhs(t, x)
    k2 = rhs(t + 0.5 * dt, x + 0.5 * dt * k1)
    k3 = rhs(t + 0.5 * dt, x + 0.5 * dt * k2)
    k4 = rhs(t + dt, x + dt * k3)
    return x + (dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)

def rollout(rhs, x0: np.ndarray, dt: float, steps: int) -> np.ndarray:
    traj = np.zeros((steps + 1, *x0.shape), dtype=np.float64)
    traj[0] = x0
    x = x0.copy()
    t = 0.0
    for k in range(steps):
        x = rk4_step(rhs, t, x, dt)
        traj[k + 1] = x
        t += dt
    return traj
```

Dataset generation

The repository trains on pointwise state/derivative pairs and evaluates on long rollouts. That is the right division for a modular rewrite too: export **both** derivative datasets and rollout datasets. The original pendulum builder caches `.npz` derivative pairs in `pendulum-cache`, with one train split and one test split; the evaluator separately constructs 1000 rollout seeds and 1000 RK4 steps. [12]

```
# src/stable_dyn/data/build_dataset.py
from __future__ import annotations

from dataclasses import asdict
import json
import numpy as np
from pathlib import Path

from stable_dyn.sim.integrators import rollout

def sample_box(low: np.ndarray, high: np.ndarray, n: int, rng:
np.random.Generator) -> np.ndarray:
    return rng.uniform(low, high, size=(n, low.shape[0]))
```

```
def build_derivative_dataset(
    system,
    train_n: int,
    val_n: int,
    state_low: np.ndarray,
    state_high: np.ndarray,
    seed: int,
    out_dir: Path,
) -> None:
    rng = np.random.default_rng(seed)
    out_dir.mkdir(parents=True, exist_ok=True)

    x_train = sample_box(state_low, state_high, train_n, rng)
    x_val = sample_box(state_low, state_high, val_n, rng)

    dx_train = system.rhs(0.0, x_train)
    dx_val = system.rhs(0.0, x_val)

    np.savez(out_dir / "train_derivs.npz", x=x_train, dx=dx_train)
    np.savez(out_dir / "val_derivs.npz", x=x_val, dx=dx_val)

def build_rollout_dataset(
    system,
    rollout_n: int,
    steps: int,
    dt: float,
    state_low: np.ndarray,
    state_high: np.ndarray,
    seed: int,
    out_dir: Path,
) -> None:
    rng = np.random.default_rng(seed)
    x0 = sample_box(state_low, state_high, rollout_n, rng)
    traj = rollout(system.rhs, x0, dt=dt, steps=steps)  # shape: [T+1, N, d]

    np.savez(out_dir / "val_rollouts.npz", x0=x0, traj=traj, dt=np.array(dt))

def write_metadata(system_name: str, system_cfg: dict, out_dir: Path) -> None:
    (out_dir / "metadata.json").write_text(json.dumps({
        "system_name": system_name,
        "system_config": system_cfg,
    }, indent=2))
```

ICNN, Lyapunov network, and stable dynamics

This is the heart of the implementation. The ICNN structure follows the 2017 ICNN paper: affine connections from the input at every layer, positive hidden-to-hidden weights on the convex path, scalar output, and convex nondecreasing activations. The stable-dynamics paper then turns that scalar convex network into a positive-definite Lyapunov function with a shift and quadratic term. The repository implements the same projection but exposes several convenience variants; the default below is the **paper-faithful** one. [13]

```
# src/stable_dyn/models/activations.py
from __future__ import annotations
import torch
from torch import nn

class SmoothedReLU(nn.Module):
    """
    Paper-faithful smooth ReLU:
      0                   if x <= 0
      x^2 / (2d)          if 0 < x < d
      x - d/2             if x >= d
    """
    def __init__(self, d: float = 1e-2):
        super().__init__()
        self.d = float(d)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.where(
            x <= 0,
            torch.zeros_like(x),
            torch.where(x < self.d, x.square() / (2.0 * self.d), x - 0.5 *
self.d),
        )


# src/stable_dyn/models/icnn.py
import torch.nn.functional as F

class ICNNScalar(nn.Module):
    """
    Scalar-valued ICNN g(x).
    x -> hidden_i via unrestricted affine maps
    z_{i-1} -> hidden_i via positive weights only
    """
    def __init__(self, in_dim: int, hidden_dims=(64, 64), activation: nn.Module
| None = None):
        super().__init__()
        self.activation = activation or SmoothedReLU(1e-2)
```

```
        h0, *rest = hidden_dims
        self.x_layers = nn.ModuleList([nn.Linear(in_dim, h0)])
        self.z_weights = nn.ParameterList()

        prev = h0
        for h in rest:
            self.x_layers.append(nn.Linear(in_dim, h))
            self.z_weights.append(nn.Parameter(torch.empty(h, prev)))
            nn.init.kaiming_uniform_(self.z_weights[-1], a=5**0.5)
            prev = h

        self.out_x = nn.Linear(in_dim, 1)
        self.out_z = nn.Parameter(torch.empty(1, prev))
        nn.init.kaiming_uniform_(self.out_z, a=5**0.5)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        z = self.activation(self.x_layers[0](x))
        for x_layer, z_weight in zip(self.x_layers[1:], self.z_weights):
            z = self.activation(x_layer(x) + F.linear(z, F.softplus(z_weight)))
        return self.out_x(x) + F.linear(z, F.softplus(self.out_z))


# src/stable_dyn/models/lyapunov.py
class AffineWarp(nn.Module):
    """
    Safe default warp: differentiable and invertible.
    """
    def __init__(self, mean: torch.Tensor, scale: torch.Tensor):
        super().__init__()
        self.register_buffer("mean", mean.clone().detach())
        self.register_buffer("scale", scale.clone().detach().clamp_min(1e-8))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return (x - self.mean) / self.scale


class LyapunovICNN(nn.Module):
    """
    V(x) = sigma(g(F(x)) - g(F(0))) + eps * ||x||^2
    """
    def __init__(
        self,
        in_dim: int,
        hidden_dims=(64, 64),
        eps_quad: float = 1e-3,
        smooth_d: float = 1e-2,
        warp: nn.Module | None = None,
```

```
    ):
        super().__init__()
        self.g = ICNNScalar(in_dim, hidden_dims,
activation=SmoothedReLU(smooth_d))
        self.sigma = SmoothedReLU(smooth_d)
        self.eps_quad = float(eps_quad)
        self.warp = warp or nn.Identity()
        self.register_buffer("zero", torch.zeros(1, in_dim))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        z = self.warp(x)
        z0 = self.warp(self.zero).expand(1, -1)
        g_x = self.g(z)
        g_0 = self.g(z0).expand_as(g_x)  # live g(0), not frozen at init
        return self.sigma(g_x - g_0) + self.eps_quad * x.square().sum(dim=-1,
keepdim=True)
```

The projection wrapper is almost exactly the paper's Eq. (10) and the repository's `Dynamics.forward`:

```
# src/stable_dyn/models/stable_dynamics.py
from __future__ import annotations
import torch
import torch.nn.functional as F
from torch import nn

class NominalMLP(nn.Module):
    def __init__(self, state_dim: int, hidden_dim: int = 100):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, state_dim),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


class StableDynamics(nn.Module):
    def __init__(self, f_hat: nn.Module, V: nn.Module, alpha: float = 1e-3,
denom_eps: float = 1e-12):
        super().__init__()
        self.f_hat = f_hat
        self.V = V
        self.alpha = float(alpha)
        self.denom_eps = float(denom_eps)
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x must require grad because we differentiate V wrt x
        if not x.requires_grad:
            x = x.requires_grad_(True)

        fx_hat = self.f_hat(x)
        vx = self.V(x)
        grad_v = torch.autograd.grad(vx.sum(), x, create_graph=True)[0]

        numer = F.relu((grad_v * fx_hat).sum(dim=-1, keepdim=True) + self.alpha
* vx)
        denom = grad_v.square().sum(dim=-1,
keepdim=True).clamp_min(self.denom_eps)

        return fx_hat - grad_v * (numer / denom)
```

Training and evaluation scripts

The repository's trainer is intentionally simple: DataLoader, Adam, MSE-style loss, checkpointing, and TensorBoard logging. That makes it a good baseline. I recommend preserving that simplicity but adding explicit rollout metrics and ablations over the Lyapunov architecture. The repository also has an optional `smooth_v` penalty; it is off in the public sweep, but it is worth keeping as an optional switch. [14]

```
# src/stable_dyn/training/losses.py
from __future__ import annotations
import torch

def derivative_mse(pred_dx: torch.Tensor, true_dx: torch.Tensor) ->
torch.Tensor:
    return (pred_dx - true_dx).square().mean()

def smooth_v_penalty(V, x: torch.Tensor, true_dx: torch.Tensor, dt: float =
1e-2) -> torch.Tensor:
    x_next = (x + dt * true_dx).detach().requires_grad_(True)
    return (V(x_next) - V(x)).clamp_min(0.0).mean()

def lyapunov_violation(model, x: torch.Tensor) -> torch.Tensor:
    x = x.detach().requires_grad_(True)
    fx = model(x)
    vx = model.V(x)
    grad_v = torch.autograd.grad(vx.sum(), x, create_graph=False)[0]
    return ((grad_v * fx).sum(dim=-1, keepdim=True) + model.alpha *
vx).clamp_min(0.0).mean()
```

```python
# src/stable_dyn/scripts/train.py
from __future__ import annotations
import torch
from torch.utils.data import DataLoader, TensorDataset

def train(cfg, model, train_npz, val_npz, out_dir):
    train = TensorDataset(
        torch.from_numpy(train_npz["x"]).float(),
        torch.from_numpy(train_npz["dx"]).float(),
    )
    val = TensorDataset(
        torch.from_numpy(val_npz["x"]).float(),
        torch.from_numpy(val_npz["dx"]).float(),
    )

    train_loader = DataLoader(train, batch_size=cfg.batch_size, shuffle=True)
    val_loader = DataLoader(val, batch_size=cfg.batch_size, shuffle=False)

    opt = torch.optim.Adam(model.parameters(), lr=cfg.lr,
weight_decay=cfg.weight_decay)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(opt,
T_max=cfg.epochs)

    best_val = float("inf")
    for epoch in range(cfg.epochs):
        model.train()
        for x, dx in train_loader:
            x = x.requires_grad_(True)
            pred_dx = model(x)
            loss = derivative_mse(pred_dx, dx)

            if cfg.smooth_v_weight > 0:
                loss = loss + cfg.smooth_v_weight * smooth_v_penalty(model.V,
x, dx, dt=cfg.train_dt)

            opt.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), cfg.grad_clip)
            opt.step()

        scheduler.step()

        model.eval()
        with torch.no_grad():
            val_losses = []
            for x, dx in val_loader:
                # need grad for V-based models, so only pure mse is tracked
```

```
under no_grad
                x = x.float()
                pred_dx = model.f_hat(x) if hasattr(model, "f_hat") else
model(x)
                val_losses.append(derivative_mse(pred_dx, dx.float()).item())

        mean_val = sum(val_losses) / max(1, len(val_losses))
        if mean_val < best_val:
            best_val = mean_val
            torch.save(model.state_dict(), out_dir / "best.pt")
```

```
# src/stable_dyn/scripts/evaluate.py
from __future__ import annotations
import numpy as np
import torch
import matplotlib.pyplot as plt

def circular_difference(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    d = a - b
    return (d + np.pi) % (2 * np.pi) - np.pi

def rollout_torch_model(model, x0: np.ndarray, dt: float, steps: int) ->
np.ndarray:
    x = torch.from_numpy(x0).float()
    traj = [x.numpy().copy()]
    for _ in range(steps):
        x_req = x.detach().requires_grad_(True)
        k1 = model(x_req).detach()
        k2 = model((x + 0.5 * dt * k1).detach().requires_grad_(True)).detach()
        k3 = model((x + 0.5 * dt * k2).detach().requires_grad_(True)).detach()
        k4 = model((x + dt * k3).detach().requires_grad_(True)).detach()
        x = x + (dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
        traj.append(x.numpy().copy())
    return np.stack(traj, axis=0)

def plot_phase_portrait(system, model, theta_grid, omega_grid, out_png):
    T, W = np.meshgrid(theta_grid, omega_grid)
    X = np.stack([T.ravel(), W.ravel()], axis=-1)

    true_dx = system.rhs(0.0, X)
    with torch.enable_grad():
        pred_dx =
model(torch.from_numpy(X).float().requires_grad_(True)).detach().numpy()

    fig, ax = plt.subplots(figsize=(6, 5))
    ax.streamplot(T, W, true_dx[:, 0].reshape(T.shape), true_dx[:,
```

```
1].reshape(W.shape),
                  density=1.1, linewidth=0.8)
    ax.quiver(X[:, 0], X[:, 1], pred_dx[:, 0], pred_dx[:, 1], alpha=0.35)
    ax.set_xlabel("theta")
    ax.set_ylabel("omega")
    fig.tight_layout()
    fig.savefig(out_png, dpi=160)
```

Notebook layout

The notebooks should stay thin:

- `simulate_systems.ipynb` selects a system config, simulates trajectories, and plots state and phase portraits.
- `build_datasets.ipynb` generates train/validation derivative data plus rollout validation sets and inspects state coverage.
- `train_models.ipynb` trains the baseline MLP, paper-faithful stable ICNN, repo-reproduction variants, and known-energy baselines.
- `evaluate_models.ipynb` reproduces the paper-style pendulum visuals, computes long-horizon metrics, and renders ablation tables.

That notebook split matches the paper/repository workflow—simulate, fit derivatives, then compare long rollouts—while keeping all logic importable and testable. [15]

Training recipes and empirical protocol

Dataset format

I recommend four explicit artifacts per experiment:

File	Keys	Shape	Purpose
`train_derivs.npz`	`x`, `dx`	`[N, d]`, `[N, d]`	Supervised derivative learning
`val_derivs.npz`	`x`, `dx`	`[N_val, d]`, `[N_val, d]`	Model selection
`val_rollouts.npz`	`x0`, `traj`, `dt`	`[M, d]`, `[T+1, M, d]`, scalar	Long-horizon evaluation
`metadata.json`	system name, params, seeds, bounds, code version	JSON	Reproducibility

For pendulum systems, store angle coordinates already wrapped into $[-\pi, \pi)$ and compute evaluation errors with circular differences, exactly because the repository wraps angular comparisons during evaluation. [12]

Configuration examples

```yaml
# configs/systems/pendulum.yaml
name: pendulum
params:
  g: 9.81
  length: 1.0
  damping: 0.3
sampling:
  train_n: 50000
  val_n: 10000
  rollout_n: 512
  dt: 0.01
  steps: 1000
  state_low: [-3.14159265, -6.28318530]
  state_high: [ 3.14159265,  6.28318530]
seeds:
  data: 11
  train: 22
  eval: 33
```

```yaml
# configs/systems/mass_spring_damper.yaml
name: mass_spring_damper
params:
  mass: 1.0
  spring_k: 2.0
  damping_c: 0.4
sampling:
  train_n: 20000
  val_n: 4000
  rollout_n: 512
  dt: 0.01
  steps: 1000
  state_low: [-2.0, -2.0]
  state_high: [ 2.0,  2.0]
seeds:
  data: 41
  train: 42
  eval: 43
```

```yaml
# configs/models/stable_icnn.yaml
name: stable_icnn
state_dim: 2
f_hat:
  hidden_dim: 100
lyapunov:
  hidden_dims: [64, 64]
  eps_quad: 0.001
  smooth_d: 0.01
projection:
  alpha: 0.001
  denom_eps: 1.0e-12
```

Hyperparameter recipes

Recipe	Purpose	$\hat{f}$	V	Optimizer	LR	Batch	Epochs	Notable settings
Repository baseline	Reproduce naive model	`2N -> 116 -> 116 -> 2N`	none	Adam	1e-3	2000	1000	derivative MSE
Repository stable	Closest public stable recipe	`2N -> 100 -> 100 -> 2N`	projection hidden `hp=60`, default public sweep over `NN-REHU`	Adam	1e-3	2000	1000	`alpha ∈ {1e-3,1e-4}`, `projfn_eps ∈ {1e-2,5e-3,1e-3}`, `rehu ∈ {5e-3,1e-3,5e-4}`
Known-energy stable	Oracle V baseline	`2N -> 100 -> 100 -> 2N`	analytic energy	Adam	5e-3	2000	1000	pendulum-only oracle

Recipe	Purpose	$\hat{f}$	V	Optimizer	LR	Batch	Epochs	Notable settings
Paper-faithful stable ICNN	Recommended default	`d -> 100 -> 100 -> d`	ICNN $64, 64$ + live $g(0)$ + smooth outer activation	Adam	1e-3	512	300	derivative MSE first, optional rollout fine-tune
Modern robust default	Higher-fidelity experiments	`d -> 128 -> 128 -> d`	ICNN $64, 64$ or $128, 128$	AdamW	1e-3	512	300–500	cosine decay, grad clip, rollout eval every epoch

The first three rows are grounded in the public shell scripts, repository model code, and the pendulum architecture described in the manuscript. Note that the public batch size is effectively full-batch for the pendulum sizes shown in the paper, because the dataset builder creates only $200n$ samples per split while the wrappers use batch size 2000. [16]

Alternative loss functions

Loss	Formula	Source status	Strength	Weakness	Recommendation
Derivative MSE	$\mathbb{E}\|\hat{f}(x) - \dot{x}\|^2$	Paper/repo core	Simple, stable, faithful	Can underweight rollout quality	Default
Derivative MSE + `smooth_v`	MSE + $\lambda \max(V(x + \Delta t\, \dot{x}) - V(x), 0)$	Repo optional	Encourages local Lyapunov decrease	Ground-truth-step heuristic, not theorem core	Optional
One-step state loss	$\mathbb{E}\|x + \Delta t\, f(x) - x_{t+1}\|^2$ or RK4 equivalent	Engineering extension	Better match to simulator outputs	Depends on chosen integrator	Useful if you store x_{t+1}
Multi-step rollout loss	$\sum_{t=1}^{H} \|\hat{x}_t - x_t\|^2$	Engineering extension	Strong long-horizon fit	More expensive, harder optimization	Fine-tune after derivative pretraining

Loss	Formula	Source status	Strength	Weakness	Recommendation
Explicit violation penalty	$\mathbb{E}[\max(\nabla V^\top f + \alpha V, 0)]$	Diagnostic extension	Good test statistic	Redundant if projection is correct	Track, don't optimize by default

The repository's training objective for stable dynamics is derivative MSE with an optional `smooth_v` term; the public sweep sets `smooth_v=0`. Because the projection should already make violations negligible up to numerical error, I would treat the explicit violation term as a diagnostic rather than a primary objective. [17]

Evaluation metrics

Metric	Definition	Why it matters
Derivative MSE	$\mathbb{E}\|\hat{f}(x) - \dot{x}\|^2$	Reproduces training target
One-step state RMSE	RMSE after one RK4 step	Connects derivative fit to integration accuracy
Long-horizon rollout RMSE	RMSE over full trajectories	Most important physical fidelity metric
Circular angle RMSE	Wrapped angular error for pendulum coordinates	Correct treatment of periodic states
Lyapunov violation rate	fraction of samples with $\nabla V^\top f + \alpha V > \tau$	Numerical stability check
Lyapunov monotonicity ratio	fraction of rollout steps with $V(x_{t+1}) \le V(x_t) + \tau$	Empirical proxy for non-expansion
Final-state norm decay	$\|x_T\|/\|x_0\|$ and semilog slope	Stability behavior
Energy trend	true energy or analytic baseline over rollout	Useful on pendulum/MSD with known energies

The repository's evaluator currently computes long-horizon state error with RK4 and circular angle handling, but it does not actually use the energy function it instantiates. A better evaluation notebook should keep the existing rollout error plot and add Lyapunov/energy trend plots explicitly. [18]

Below is the requested mermaid diagram for training and dataflow.

```
flowchart TD
    A[Sample states x] --> B[Nominal network f_hat(x)]
    A --> C[Lyapunov network V(x)]
    C --> D[autograd grad V(x)]
    B --> E[Half-space projection]
```

```
    D --> E
    E --> F[Stable derivative f(x)]
    F --> G[Derivative loss]
    A --> H[RK4 rollout]
    H --> I[Rollout metrics]
    C --> J[Lyapunov metrics]
    G --> K[Optimizer]
    K --> B
    K --> C
```

Example experiments

For the physical benchmark that comes from the source, the evaluation target is straightforward: reproduce the paper's pendulum outputs. For $n = 1$, generate a streamplot of the true vector field, a streamplot or quiver plot of the learned field, a sample trajectory overlay, and a contour plot of the learned V. For $n \in \{1, 2, 4, 6, 8\}$, compute cumulative rollout error over 1000 RK4 steps and compare the stable model against the naive MLP baseline. The paper reports that the stable model tracks the simple pendulum well and, for the 8-link case, eventually exhibits decreasing error later in the rollout while the naive model's error continues to grow. [19]

For the second physical system, use a mass-spring-damper benchmark with known stable linear dynamics and a known quadratic energy. This makes a very strong debugging problem because you can compare the analytic energy trend, the learned V, the projection-violation rate, and the long-horizon decay of $\|x_t\|$. I recommend plotting position/velocity time traces, a phase portrait, semilog norm decay, and $V(x_t)$ over time.

Testing, reproducibility, and ambiguities

Unit tests

Test	What it checks	Why it matters
`test_system_rhs_shapes`	all systems return shape `[batch, d]`	interface sanity
`test_rk4_small_dt_consistency`	RK4 matches a high-precision reference for small dt	simulation correctness
`test_icnn_jensen`	numerical Jensen inequality on random samples	catches broken convexity in ICNN implementation
`test_lyapunov_origin`	$V(0) \approx 0$ and $V(x) > 0$ for sampled $x \neq 0$	positive-definiteness sanity
`test_projection_constraint`	$\nabla V^\top f + \alpha V \leq \tau$ on random batches	core theorem check

Test	What it checks	Why it matters
`test_no_nan_backward`	backward pass through projection and `grad(V,x)` is finite	training robustness
`test_pendulum_angle_wrap`	circular error stays in $[-\pi, \pi)$	correct pendulum evaluation
`test_known_energy_baseline`	oracle V is nonnegative and minimized at origin	baseline integrity

A minimal but high-value projection test looks like this:

```
# tests/test_stable_projection.py
import torch

def test_projection_enforces_lyapunov_halfspace(stable_model):
    x = torch.randn(128, stable_model.f_hat.net[-1].out_features,
requires_grad=True)
    fx = stable_model(x)
    vx = stable_model.V(x)
    grad_v = torch.autograd.grad(vx.sum(), x, create_graph=False)[0]
    lhs = (grad_v * fx).sum(dim=-1, keepdim=True) + stable_model.alpha * vx
    assert torch.all(lhs <= 1e-6)
```

Reproducibility instructions

The public repository does not set seeds in its trainer or dataset builder, and the manuscript does not publish a complete pendulum hyperparameter sheet. A serious reimplementation should therefore treat reproducibility as a first-class feature: separate data/train/eval seeds, save the full config for every run, record package versions, and hash the exported datasets. [20]

```
# src/stable_dyn/utils/seeding.py
from __future__ import annotations
import os, random
import numpy as np
import torch

def seed_everything(seed: int) -> None:
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
```

```
    torch.use_deterministic_algorithms(True, warn_only=True)
    torch.backends.cudnn.benchmark = False
```

A conservative modern environment for the physical-systems-only package is:

```
# pyproject.toml sketch
[project]
name = "stable-icnn-dynamics"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
  "torch>=2.2",
  "numpy>=1.26",
  "scipy>=1.11",
  "sympy>=1.12",
  "matplotlib>=3.8",
  "pyyaml>=6.0",
  "tensorboard>=2.15"
]

[project.optional-dependencies]
dev = ["pytest>=8.0", "jupyterlab>=4.0", "pandas>=2.2", "ipykernel>=6.0"]
```

That modern stack deliberately omits the original repository's image/video dependencies, along with TensorFlow, because the state-space pendulum/stable-dynamics path is torch-based. [21]

Ambiguities and assumptions

The source leaves several points ambiguous enough that they should be documented explicitly in the implementation itself.

Ambiguity or caveat	Why it matters	Resolution used here
Exact pendulum reproduction settings are not fully documented in the paper	You cannot fully audit a reproduction from the manuscript alone	Treat the public shell scripts as the closest available baseline
Public sweep uses `NN-REHU`	Not a strict ICNN, contrary to the paper's ICNN-based description	Use paper-faithful ICNN as default; keep `NN-REHU` only as ablation
`MakePSD` stores `g(0)` once at initialization	Diverges from the paper's $g(x) - g(0)$ construction during training	Recompute live $g(0)$ every forward pass

Ambiguity or caveat	Why it matters	Resolution used here
`PosDefICNN` differs from the manuscript's Eq. (12)	Useful engineering variant, but not exactly the same theorem path	Treat as repo-faithful variant, not default
Optional warp F in paper must be invertible	Non-invertible feature maps can break the proof	Allow only affine or otherwise explicitly invertible warps in the Lyapunov branch
Repository `scale_fx` option is not a dependable reference implementation	It is configured in a way that is easy to misread and should not be copied blindly	Omit from the default package until justified and tested

These caveats are evidenced directly by the paper equations, the repository code, and the public issue threads asking about the unresolved projection-family choice and missing reproduction parameters. My working assumptions for the modular redesign are therefore: autonomous continuous-time systems, full-state observation, paper-faithful stable ICNN as the default, repo-faithful variants as ablations, pendulum as the source benchmark, and mass-spring-damper as the second requested physical example. [22]

---

[1] [3] [4] [9] [10] [15] [19] [22] Learning Stable Deep Dynamics Models
https://papers.neurips.cc/paper/9292-learning-stable-deep-dynamics-models.pdf

[2] [8] GitHub - locuslab/stable_dynamics: Companion code to "Learning Stable Deep Dynamics Models" (Manek and Kolter, 2019) · GitHub
https://github.com/locuslab/stable_dynamics

[5] [6] [7] [17] raw.githubusercontent.com
https://raw.githubusercontent.com/locuslab/stable_dynamics/master/models/stabledynamics.py

[11] [18] raw.githubusercontent.com
https://raw.githubusercontent.com/locuslab/stable_dynamics/master/pendulum_error.py

[12] raw.githubusercontent.com
https://raw.githubusercontent.com/locuslab/stable_dynamics/master/datasets/pendulum.py

[13] Input Convex Neural Networks
https://proceedings.mlr.press/v70/amos17b/amos17b.pdf

[14] [20] raw.githubusercontent.com
https://raw.githubusercontent.com/locuslab/stable_dynamics/master/train.py

[16] raw.githubusercontent.com
https://raw.githubusercontent.com/locuslab/stable_dynamics/master/train_pendulum_simple

[21] stable_dynamics/requirements.txt at master · locuslab/stable_dynamics · GitHub
https://github.com/locuslab/stable_dynamics/blob/master/requirements.txt